Word2Vec

Word2Vec is a *word embedding technique* proposed by **Tomas Mikolov et al. (2013)** that represents words as dense, low-dimensional numerical vectors such that words with similar meanings have similar vector representations.

Definition

Word2Vec is a shallow neural network model that learns vector representations of words based on their contextual usage in a large corpus.

Key Idea

"A word is known by the company it keeps." (Distributional Semantics)

Words appearing in similar contexts will have similar embeddings.

Architectures of Word2Vec

1. CBOW (Continuous Bag of Words)

- **Predicts the target word from surrounding context words**
- Faster and works well with large datasets

Example:
Context: *I love deep* ___
Target: *learning*

✅ Uses average of context vectors
✅ Suitable when training data is large

---

2. Skip-Gram

- **Predicts surrounding context words from the target word**
- Performs better for rare words

Example:
Target: *learning*
Context: *deep, neural*

✅ Better semantic accuracy
✅ Slower than CBOW

Training Objective

Word2Vec maximizes the probability:

$P(w_{context} \mid w_{target})$

using:

- **Negative Sampling** or
- **Hierarchical Softmax**
  to reduce computation cost

Features of Word2Vec

- Produces **dense embeddings**
- Captures **semantic relationships**
- Supports **vector arithmetic**

Example:

```
king − man + woman ≈ queen
```

Advantages

- Efficient for large corpora
- Captures semantic similarity
- Useful for many NLP tasks

Limitations

- Context-independent (same word → same vector)

- Cannot handle polysemy well
- Requires large training data

(Solved by models like GloVe, FastText, ELMo, BERT)

Applications

- Information Retrieval
- Sentiment Analysis
- Text Classification
- Machine Translation
- Recommendation Systems

Conclusion (Exam-Ready)

Word2Vec is an efficient neural embedding technique that converts words into meaningful vector representations by learning from surrounding contexts using CBOW and Skip-Gram architectures. It significantly improves semantic understanding in NLP tasks.

BERT (Bidirectional Encoder Representations from Transformers)

BERT is a **pre-trained language model** introduced by **Google** that converts natural language text into **context-aware embedding vectors** using the **Transformer** architecture.

Can you say it like Word2Vec?

Yes ✓ — but with an important improvement.

Simple statement (correct)

"BERT is a model that converts natural language text into embedding vectors."

Better / exam-ready statement ✓✓

“BERT is a deep learning–based language model that converts words and sentences into contextual embedding vectors by considering both left and right context.”

Key Characteristics of BERT

1. Bidirectional Context

- Unlike Word2Vec, BERT reads text **from left to right and right to left simultaneously**
- Meaning of a word changes based on context

Example:

- *bank* in “river bank”
- *bank* in “money bank”

☞ BERT gives **different embeddings**, Word2Vec gives **same embedding**

---

2. Transformer Encoder

- Uses **self-attention mechanism**
- Captures long-range dependencies efficiently

---

3. Pre-training Tasks

a) Masked Language Model (MLM)

- Random words are masked
- Model predicts them using surrounding context

b) Next Sentence Prediction (NSP)

- Learns sentence-to-sentence relationships

Comparison with Word2Vec (Short)

Feature	Word2Vec	BERT
Context	Context-independent	Context-dependent

Model type	Shallow NN	Deep Transformer
Handles polysemy	❌ No	✅ Yes
Output	Word embeddings	Word + sentence embeddings

Applications of BERT

- Question Answering
- Sentiment Analysis
- Named Entity Recognition
- Text Classification
- Search Engines (Google Search)

Conclusion (Exam-Friendly)

BERT is a bidirectional Transformer-based language model that generates deep contextual embeddings of words and sentences, enabling better semantic understanding compared to traditional word embedding techniques like Word2Vec.

Why Histograms Are Problematic (Even for One Variable)

A histogram seems like a simple way to estimate a probability distribution, but it has **serious limitations**, especially for continuous data.

1. Histograms Overfit the Training Data

A histogram essentially memorizes the training samples.
This means the model does not learn the *true* underlying distribution—only the exact pattern present in the seen data.

As a result:

- The learned histogram ≈ training data distribution
- It fails to generalize to new data

---

2. Zero Probability Between Bins

Histograms assign probability only to **bins**.

So if a value falls **between bin centers**, the histogram gives it:

$p(x) = 0$

This is problematic because:

- Real continuous distributions are smooth
- Probability should not drop to zero in gaps where no training samples happened to appear

---

3. Poor Generalization

Because histograms rely on rigid bins:

- Slight changes in x can cause abrupt probability jumps
- Unseen values are treated as impossible
- The model cannot capture smooth trends in the data

Intuitive Example

Suppose your training data values are:
2.1, 2.2, 2.3
and the histogram bins are:
[2.0–2.2], [2.2–2.4]

Then:

- Value 2.15 → falls inside a bin → gets some probability
- Value 2.18 → also gets probability
- Value 2.25 → different bin → different probability
- Value 2.17 (if no sample in that exact small region) → might get near-zero probability

Even though all these values are almost the same in reality, the histogram cannot reflect that smoothness.

Summary (Short, for Exams)

- Histograms **overfit** because they memorize training data.
- They give **zero probability** to unseen values between bins.
- Their piecewise-constant nature leads to **poor generalization** for continuous variables.
- Therefore, even for 1D data, histograms are a **bad density model**.

Family 1: RNN-based Autoregressive Models

Core Idea

RNN-based autoregressive models generate data **one step at a time**, where each output depends on all previous outputs.

How It Works

1. **Hidden State (Memory)**

$$h_t = f_\theta(h_{t-1}, x_{t-1})$$

- The hidden state h_t summarizes the **entire past** $x_{<t}$
- It acts as a memory of previous inputs

1. **Prediction**

$$p_\theta(x_t \mid h_t)$$

- The model predicts the probability of the next value using the hidden state

---

Role of LSTM / GRU

- Standard RNNs suffer from **vanishing gradients**
- **LSTM** and **GRU** use gates to:
    - Preserve important information
    - Forget irrelevant history
    - Learn long-range dependencies efficiently

Why They Are Autoregressive

- At time step ttt, the model:
    - Uses only past data (x<tx_{<t}x<t)
    - Predicts xtx_txt
- This enforces the autoregressive constraint exactly

Main Limitation: Sequential Computation

✕ Cannot parallelize

- To compute hth_tht, you must have ht−1h_{t-1}ht−1
- Each step depends on the previous step

✕ Slow on GPUs

- GPUs are designed for **parallel computation**
- RNNs force **step-by-step processing**

Resulting Problem

- Training and sampling are slow
- Not efficient for very long sequences

Motivation for Other Models

This limitation motivates newer autoregressive families (e.g., masked CNNs, Transformers) that:

- Maintain autoregressive structure
- **Avoid sequential computation**
- Enable parallel training

One-Line Exam Summary

RNN-based autoregressive models use a hidden state to summarize past inputs and predict the next value sequentially, but their strictly sequential nature makes them slow and hard to parallelize.

Family 2: MADE — The Key Insight

Problem with RNN-based Autoregressive Models

- RNNs enforce the autoregressive order **through sequential computation**
- Step ttt cannot be computed without step t−1t-1t−1
- This prevents **parallel training** and makes models slow on GPUs

☞ **Key question:**
Can we enforce the autoregressive constraint **without sequential computation**?

What is MADE?

MADE (Masked Autoencoder for Distribution Estimation) is an autoregressive model that:

- Uses a **standard feed-forward MLP**
- Enforces autoregressive dependencies using **binary masks on weights**

So instead of *computation order*, MADE uses *masking*.

Core Idea of MADE

Problem with a Normal MLP

In a regular MLP:

- Each output x^i\hat{x}_ix^i depends on **all input variables**
- This violates autoregressive modeling because x^i\hat{x}_ix^i should only depend on x<ix_{<i}x<i

---

How MADE Fixes This: Masking Weights

Step 1: Assign Order Indices

- Each input and hidden unit is assigned an index:

m∈{1,2,…,D}m \in \{1, 2, \dots, D\}m∈{1,2,…,D}

This index represents **which variable order it belongs to**

Step 2: Masking Rule

A connection from unit jjj to unit kkk is allowed **only if**:

mj≤mkm_j \le m_kmj≤mk

❌ If mj>mkm_j > m_kmj>mk: connection is removed (masked to zero)

This ensures:

- No unit can depend on **future variables**

Step 3: Effect on Outputs

- Output x^i\hat{x}_ix^i can only see inputs:

x1,x2,…,xi−1x_1, x_2, \dots, x_{i-1}x1,x2,…,xi−1

- Autoregressive constraint is strictly enforced

Key Result

✅ **All conditional probabilities**

p(xi∣x<i)for all ip(x_i \mid x_{<i}) \quad \text{for all } ip(xi∣x<i)for all i

are computed in **one forward pass**

✅ Training is **fully parallel**

✅ Likelihood is **exact**, like RNNs

Why MADE Is Better Than RNNs (Training)

Aspect	RNN	MADE
Autoregressive constraint	Sequential computation	Weight masking
Training	Sequential	Fully parallel
GPU efficiency	Poor	High
Likelihood	Exact	Exact

Order-Agnostic Training (Extra Advantage)

- The variable ordering is **randomly changed**
- Different random masks are used during training
- This creates an **ensemble of orderings**

✅ Model does not rely on a fixed ordering
✅ Better generalization

Intuition (One Line)

MADE enforces autoregressive dependencies by masking network connections instead of relying on sequential computation.

One-Paragraph Exam Answer

MADE is an autoregressive density estimator that replaces sequential RNN computation with masked connections in a feed-forward neural network. By assigning order indices to units and masking weights so that future inputs are blocked, each output depends only on past variables. This allows all conditional probabilities to be computed in a single forward pass, enabling fully parallel training while maintaining exact likelihood estimation.

1. What is MLP (Multi-Layer Perceptron)?

Simple idea

An **MLP** is a **basic neural network** used to learn patterns from data.

How it looks

- Input layer
- One or more **hidden layers**
- Output layer

Each layer:

- Takes numbers
- Multiplies by weights
- Applies an activation function

Example

Input: exam marks
Output: pass / fail

The MLP learns **how inputs map to outputs**.

Key point

✅ MLP can learn complex functions
❌ A normal MLP lets every output see **all inputs**

---

2. What is MLE (Maximum Likelihood Estimation)?

Simple idea

MLE is a training rule.

It means:
"Adjust the model so that the data we saw becomes as probable as possible."

In one sentence

MLE chooses model parameters that **maximize the probability of the training data**.

Example

If your model predicts:

- A with probability 0.9 ✅ → good

- A with probability 0.1 ❌ → bad

MLE training **rewards high probability for correct answers** and **penalizes low probability**.

Mathematically:

MLE=max⁡log⁡p(x)\text{MLE} = \max \log p(x)MLE=maxlogp(x)

In practice:

- We **minimize negative log-likelihood (NLL)**

Key point

✅ Standard way to train generative models
✅ Used for RNNs, MADE, Transformers, etc.

---

3. What is MADE (Masked Autoencoder for Distribution Estimation)?

Problem it solves

- A normal MLP allows outputs to depend on **future inputs**
- This breaks autoregressive modeling

Idea of MADE

Use an **MLP**, but **block some connections** so rules are obeyed.

These blocks are done using **masks**.

What does "masking" mean?

- Some weights are set to **zero**
- This prevents information from flowing where it shouldn't

So:

- Output for x1x_1x1 sees nothing

- Output for x2x_2x2 sees x1x_1x1
- Output for x3x_3x3 sees x1,x2x_1, x_2x1,x2

✅ Autoregressive rule enforced
✅ Still uses a normal MLP

Why is MADE important?

Compared to RNNs:

- RNNs are **slow** (step-by-step)
- MADE computes **all outputs at once**

✅ Fully parallel
✅ Fast on GPUs
✅ Exact likelihood

---

4. One-line summaries (easy to memorize)

- **MLP**: A basic feed-forward neural network.
- **MLE**: A training method that makes the model assign high probability to seen data.
- **MADE**: A masked MLP that enforces autoregressive dependencies without sequential computation.

---

5. Simple comparison table

Term	What it is	Purpose
MLP	Neural network	Learn mappings
MLE	Training principle	Learn probabilities
MADE	Masked MLP model	Fast autoregressive density estimation

---

6. Exam-ready paragraph

An MLP is a standard feed-forward neural network used for function approximation. MLE is a training principle that learns model parameters by maximizing the likelihood of the observed data. MADE is an autoregressive generative model that uses masked connections in an MLP to enforce dependency ordering, allowing exact likelihood computation with fully parallel training.

Family 3: Masked Convolutions — Adding Structure

Why do we need this family?

- **MADE** works well for **1-D data** (like simple sequences)
- But **images and audio** have strong **local structure**:
    - Nearby pixels are related
    - Nearby audio samples are related

☞ We want a model that:

- Respects **autoregressive (causal) order**
- **Exploits local patterns**
- Trains **in parallel**

This leads to **masked convolutions**.

Why Convolutions?

What convolutions do well

- Capture **local patterns**
- Efficient and highly parallel
- Perfect for images and audio

Problem with standard convolutions

❌ A normal convolution looks at:

- Past
- Present
- **Future values**

But autoregressive modeling requires:

xt must depend only on x<tx_t \text{ must depend only on } x_{<t}xt must depend only on x<t

So standard convolutions are **not causal**.

The Fix: Masked Convolutions

Key idea

Zero out (mask) filter weights that look into the future

This ensures:

- The convolution only sees **past values**
- Autoregressive constraint is enforced

Case 1: 1-D Masked Convolutions (Audio)

Think of audio as a time series:

x1,x2,x3,…x_1, x_2, x_3, \dotsx1,x2,x3,…

Rule

- At time ttt, the filter can see:
  - xt−1,xt−2,…x_{t-1}, x_{t-2}, \dotsxt−1,xt−2,…
- It **cannot see**:
  - xtx_txt (sometimes)
  - xt+1,xt+2,…x_{t+1}, x_{t+2}, \dotsxt+1,xt+2,…

This is called **causal convolution**.

✅ Used in models like **WaveNet**

Case 2: 2-D Masked Convolutions (Images)

Images are treated as sequences using **raster scan order**:

→ → →
→ → →
→ → →

Rule for a pixel

A pixel can depend on:

- Pixels **to the left**
- Pixels **above**

It cannot depend on:

- Pixels **to the right**
- Pixels **below**

This keeps the autoregressive ordering correct.

✅ Used in **PixelCNN**

Mask Types: Mask A and Mask B

Mask A (Strict Mask)

- Used in the **first layer**
- The filter **cannot see the current pixel**

✅ Ensures:

p(xi) does not depend on xip(x_i) \text{ does not depend on } x_ip(xi) does not depend on xi

---

Mask B (Relaxed Mask)

- Used in **later layers**
- The filter **can see the current pixel**

Why?

- The first layer already ensures causality
- Later layers can safely build richer features

✅ Allows information accumulation without breaking autoregression

---

Receptive Field and Dilation

What is receptive field?

- The region of the input that affects an output

Problem

- Small kernels → limited context

Solution: Dilated Convolutions

- Skip input positions
- Rapidly increase receptive field

Example:

- Layer 1: sees neighbors
- Layer 2: sees farther points
- Layer 3+: sees long-range context

✅ Context grows **exponentially with depth** ✅ Essential for long-range dependencies (audio, images)

Summary Table

Concept	Simple Meaning
Masked convolution	Convolution that cannot see future values
Raster scan	Row-by-row pixel ordering
Mask A	Hides current pixel (first layer)
Mask B	Allows current pixel (later layers)
Dilation	Expands context without large kernels

One-Paragraph Exam Answer

Masked convolutions enforce the autoregressive constraint while exploiting local structure in images and audio. By masking convolutional filters, future positions are blocked, ensuring causality. In 1-D data, only past time steps are visible, while in 2-D images, pixels to the right and below are masked using raster scan order. Mask A is used in the first layer to exclude the current position, and Mask B is used in later layers to allow feature accumulation. Dilated convolutions expand the receptive field efficiently, enabling long-range dependencies.

One-Line Memory Trick

RNN = order by computation, MADE = order by masking MLP, Masked CNN = order + local structure

What does “causal” mean?

Causal means:

The present should depend only on the past, not the future.

In sequence modeling (time, audio, text, pixels):

- At time step ttt, the model **must not look ahead** to t+1,t+2,…t+1, t+2,\dotst+1,t+2,…
- Otherwise, it is **cheating**

What is a standard convolution?

A standard convolution applies a filter **symmetrically** around a position.

Example (1-D convolution)

For a signal:

x1,x2,x3,x4,x5x_1, x_2, x_3, x_4, x_5x1,x2,x3,x4,x5

A standard convolution with kernel size 3 at position x3x_3x3 uses:

[x2, x3, x4][x_2,\; x_3,\; x_4][x2,x3,x4]

So the output at position 3 depends on:

- past → x2x_2x2
- present → x3x_3x3
- **future → x4x_4x4 ❌**

Why is this not causal?

Because:

- The model is using **future information** (x4x_4x4)
- When predicting x3x_3x3, x4x_4x4 is **not supposed to be known yet**

☞ This breaks the autoregressive rule:

xt must depend only on x<tx_t \text{ must depend only on } x_{<t}xt must depend only on x<t

So we say:

Standard convolutions are not causal

Audio intuition (very clear)

Imagine predicting the **next audio sample**:

- At time ttt, you only have audio up to time $t-1$t-1t−1
- A standard convolution would also use samples from the **future**
- That is impossible in real-time generation

❌ Unrealistic
❌ Incorrect for generative models

Image intuition (2-D case)

Images are generated pixel by pixel (raster scan):

```
→ → →
→ → ?
? ? ?
```

When predicting the ? pixel:

- Allowed: pixels to the **left** and **above**
- Not allowed: pixels to the **right** or **below**

Standard convolution would look **everywhere**, including future pixels ❌
So it is **not causal** for image generation.

The fix: Causal / Masked Convolutions

To make convolutions causal:

- We **zero out (mask)** kernel weights that look into the future
- The filter sees only valid past locations

✅ This enforces the autoregressive constraint
✅ Used in PixelCNN, WaveNet, etc.

One-line exam definition ✅

A standard convolution is not causal because its output at a position depends on future input values, which violates the autoregressive requirement that predictions depend only on past data.

Tiny summary

Term	Meaning
Causal	Uses only past information
Non-causal	Uses past + future
Standard convolution	❌ Non-causal
Masked/Causal convolution	✅ Causal

1. Masked Convolutions (Core Idea)

Why masking is needed

In **generative models**, data must be generated **one step at a time**:

- Audio → sample by sample
- Images → pixel by pixel

So when predicting:

- Current value **must not depend on future values**

This is called **causality**.

Problem with standard convolution

A normal convolution looks at:

- Past
- Present
- **Future ❌**

So it **breaks causality**.

Masked Convolution (Fix)

Masked convolution:

- **Zeros out filter weights** that look into the future
- Ensures each prediction uses **only allowed past information**

✅ Autoregressive
✅ Parallel computation
✅ Exact likelihood

---

2. WaveNet (Masked Conv for Audio)

What is WaveNet?

WaveNet is a **generative model for audio**, developed by DeepMind.

Goal:

Predict the next audio sample given previous samples

Why convolution for audio?

Audio is:

- 1-D time sequence
- Strong local dependencies

Convolutions:

- Capture local patterns well
- Are fast and parallel

Causal Convolution (WaveNet)

Meaning

At time ttt, the model sees:

- xt−1,xt−2,…x_{t-1}, x_{t-2}, \dotsxt−1,xt−2,…
- **NOT** xtx_txt or future values

This is done by **masking future positions** in the convolution filter.

Dilated Convolution (Very important)

Problem

Small filters → short memory

Solution: Dilation

Dilated convolution:

- Skips inputs
- Expands receptive field quickly

Example:

- Layer 1: sees nearby samples
- Layer 2: sees farther samples
- Layer 3: sees very far samples

✅ Long-range audio structure
✅ Efficient (no huge filters)

Summary of WaveNet

- 1-D masked causal convolutions

- Dilated convolutions for long context
- Generates **very realistic speech and music**
- Sampling is slow (sample-by-sample)

---

3. PixelRNN (Images with RNNs)

Idea

Treat an image as a **sequence of pixels** in raster order:

→ → →
→ → →
→ → →

Use RNNs to model:

p(xi,j|previous pixels)p(x_{i,j} \mid \text{previous pixels})p(xi,j|previous pixels)

How PixelRNN works

- Uses RNN/LSTM
- Hidden state remembers past pixels
- Predicts current pixel distribution

✅ Exact likelihood
❌ Very slow (pixel-by-pixel RNN)

---

4. PixelCNN (Masked Conv for Images)

Motivation

PixelRNN is slow → use **convolutions instead of RNNs**

Key idea of PixelCNN

- Use **2-D masked convolutions**

- Enforce autoregressive order via masking

How masking works in images

When predicting a pixel:

- ✅ Can see pixels **to the left**
- ✅ Can see pixels **above**
- ❌ Cannot see pixels to the right or below

Mask types in PixelCNN

Mask A (First layer)

- Cannot see **current pixel**
- Ensures strict causality

Mask B (Later layers)

- Can see **current pixel**
- Allows feature reuse

Advantages of PixelCNN

✅ Fully parallel training
✅ Exact likelihood
✅ Exploits local image structure

❌ Sampling still sequential (pixel by pixel)

---

5. Gated PixelCNN (Improvement)

Problem with basic PixelCNN

- Limited ability to model complex interactions

- Vanishing gradient issues

Gated PixelCNN (Solution)

Uses **gated activation functions**:

tanh⁡(⋅)⊙σ(⋅)\tanh(\cdot) \odot \sigma(\cdot)tanh(⋅)⊙σ(⋅)

This:

- Improves gradient flow
- Increases expressiveness

✅ Better image quality
✅ More stable training

---

6. PixelCNN++ (Further Improvements)

PixelCNN++ introduced **important practical upgrades**.

Why PixelCNN++ was needed

Original PixelCNN had issues:

- Poor modeling of continuous pixel values
- Limited output distribution

Key improvements in PixelCNN++

1. Better output distribution

Uses:

- **Mixture of logistic distributions** Instead of:
- Simple softmax on 256 values

✅ Smoother pixel modeling
✅ Better likelihood

---

2. Fewer parameters

- Removes unnecessary dependencies
- More efficient architecture

---

3. Better conditioning

Models:

- Pixel intensity and color channel dependencies better

Result

✅ Much better image quality
✅ Strong benchmark model for image generation

---

7. Softmax Sampling (Final Step)

What is happening during sampling?

At each step, the model outputs:

- A **probability distribution** over possible values

Example (vocabulary or pixel values):

```
A: 0.6
B: 0.3
C: 0.1
```

Softmax sampling

- Uses **softmax** to ensure probabilities sum to 1
- Sample randomly according to probabilities

This introduces:

- Variety
- Natural randomness

Why not always choose max probability?

That would be **greedy**:

- Leads to boring, repetitive outputs

Sampling gives: ✓ Diverse images/audio
✓ More realistic generations

---

8. Big Picture Summary

Model	Data	Main Idea
WaveNet	Audio	Causal dilated 1-D conv
PixelRNN	Image	RNN over pixels
PixelCNN	Image	Masked 2-D conv
Gated PixelCNN	Image	Gating improves modeling
PixelCNN++	Image	Better distributions & efficiency

One-line memory trick □

RNN = sequential order by computation
MADE = order by masking MLP
PixelCNN/WaveNet = order + locality via masked convolutions

What Do We Want from a Density Model?

We are given **continuous high-dimensional data**

$x \in \mathbb{R}^n$

such as **images, audio, and sensor readings**.
The goal of density modeling is to learn a parameterized probability distribution

$p_\theta(x)$

that approximates the true data distribution $p_{\text{data}}(x)$.

A good density model should satisfy the following **four desiderata**:

---

1. Good Fit to Data

The model distribution should closely match the true data distribution.

- Achieved by **maximum likelihood estimation (MLE)**:

$$\max_\theta \sum_i \log p_\theta(x^{(i)})$$

- Ensures the model assigns **high probability to observed data**
- This is the primary training objective of most density models

---

2. Efficient Evaluation

For a new datapoint x:

- We should be able to **compute $p_\theta(x)$ efficiently**
- This is important for:
    - Likelihood-based evaluation
    - Anomaly detection
    - Model comparison

---

3. Efficient Sampling

The model should allow us to **generate new samples**:

$x \sim p_\theta(x)$

- Sampling must be computationally feasible
- Necessary for tasks like:
    - Image generation
    - Data augmentation
    - Simulation

---

4. Meaningful Latent Representation

The model should learn a latent variable:

$z = f_\theta(x)$

- z should be **compact and semantically meaningful**
- Useful for:
    - Representation learning
    - Interpolation
    - Downstream tasks (classification, clustering)

Why Not Mixtures of Gaussians (MoG)?

A Gaussian Mixture Model defines:

$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$

Advantages

- Sampling is easy:
    1. Choose mixture component
    2. Sample Gaussian noise
- Likelihood is tractable

Limitations (Especially for Images)

- **Realistic samples only resemble cluster centers**
- High-dimensional data (images) require:
    - A huge number of components
- Number of parameters grows with the number of modes
- Memorization rather than true generative understanding

✅ Hence, MoGs are **not generative in a deep or compositional sense**

Why Not a Plain Neural Network $x \rightarrow p_\theta(x)$?

A standard neural network can output arbitrary values, but:

Key Problems

- **Probability constraints are not guaranteed**:
  - Must ensure:

    $$p_\theta(x) \ge 0 \quad \text{and} \quad \int p_\theta(x)\,dx = 1$$

- These constraints are **not automatic** in vanilla neural networks
- Computing the normalization constant (partition function) is often intractable

❌ Therefore, a naïve neural network **cannot directly model densities**

Motivation for Modern Density Models

These limitations motivate structured models such as:

- **Variational Autoencoders (VAEs)** – latent variable models
- **Normalizing Flows** – exact likelihood + invertible mappings
- **Autoregressive models** – tractable likelihood via factorization
- **Diffusion models** – powerful sampling via gradual denoising

All of these are designed to satisfy the four desiderata **simultaneously**.

One-line Exam Summary

A good density model must fit the data well, allow efficient likelihood computation and sampling, and learn a meaningful latent representation—requirements that are not adequately met by mixtures of Gaussians or plain neural networks.

The Flow Idea (Normalizing Flows)

Core Insight

Instead of directly parameterizing a complex density $p_\theta(x)$, **normalizing flows** learn an **invertible transformation**:

$$z = f_\theta(x), \quad z \sim p_Z(z) = \mathcal{N}(0, I)$$

This maps complex data x to a simple, known distribution (standard Gaussian).

How a Flow Model Works

1. Training (Encoding: $x \rightarrow z$)

- Data samples $x \sim p_{\text{data}}(x)$ are passed through the invertible function f_θ
- The resulting z should look like samples from $\mathcal{N}(0, I)$
- Parameters θ are learned by **maximum likelihood**

---

2. Sampling (Decoding: $z \rightarrow x$)

- Sample latent variables:

$$z \sim \mathcal{N}(0, I)$$

- Generate data by applying the inverse map:

$$x = f_\theta^{-1}(z)$$

✅ Enables **exact and efficient sampling**

---

3. Evaluation (Exact Likelihood Computation)

The likelihood of x is computed using the **change-of-variables formula**:

$$\log p_\theta(x) = \log p_Z(f_\theta(x)) + \log \left| \det \left(\frac{\partial f_\theta}{\partial x} \right) \right|$$

- First term: likelihood under simple base distribution
- Second term: volume change due to transformation

✅ Provides **exact, tractable likelihoods**

Requirements on the Transformation fθf_\thetafθ

To satisfy training, sampling, and evaluation goals, fθf_\thetafθ must have the following properties:

1. Invertibility

fθ−1 must exist and be computablef_\theta^{-1} \text{ must exist and be computable}fθ−1 must exist and be computable

- Required for:
  - Sampling (z→xz \rightarrow xz→x)
  - Exact density evaluation

---

2. Differentiability

∂fθ∂x must exist\frac{\partial f_\theta}{\partial x} \text{ must exist}∂x∂fθ must exist

- Needed to compute gradients during training
- Enables Jacobian computation

---

3. Tractable Jacobian Determinant

det⁡(∂fθ∂x) must be cheap to compute\det\left(\frac{\partial f_\theta}{\partial x}\right) \text{ must be cheap to compute}det(∂x∂fθ) must be cheap to compute

- Direct Jacobian computation is O(n3)O(n^3)O(n3) → infeasible
- Flow architectures (e.g., coupling layers) are designed to make the determinant computation **efficient**

Why This Is Powerful

Normalizing flows **simultaneously satisfy all four density-model desiderata**:

Requirement	Flow Models
Good fit	Exact MLE
Evaluation	Exact likelihood
Sampling	Efficient via inverse
Latent space	Explicit, meaningful zzz

One-Line Exam Summary

Normalizing flows model complex data distributions by learning an invertible, differentiable transformation from data space to a simple base distribution, enabling exact likelihood evaluation, efficient sampling, and meaningful latent representations.

CDF as a Universal 1-D Flow

Question

Can every smooth 1-D probability density p(x)p(x)p(x) be represented by a normalizing flow?

Answer

✅ **Yes.**
In one dimension, the **Cumulative Distribution Function (CDF)** itself is always a valid normalizing flow.

---

1. Definition of the CDF Flow

For a 1-D continuous random variable xxx with density pθ(x)p_\theta(x)pθ(x), define:

fθ(x)=Fθ(x)=∫−∞xpθ(t) dtf_\theta(x) = F_\theta(x) = \int_{-\infty}^{x} p_\theta(t)\,dtfθ(x)=Fθ(x)=∫−∞xpθ(t)dt

This maps:

x ⟶ z=Fθ(x)x \;\longrightarrow\; z = F_\theta(x)x⟶z=Fθ(x)

---

2. What Distribution Does zzz Follow?

A fundamental result from probability theory:

$z = F_\theta(x) \;\sim\; \text{Uniform}[0,1]$

✅ This means the CDF **transforms any continuous distribution into a uniform distribution**.

---

3. Why Is the CDF a Valid Normalizing Flow?

A normalizing flow must satisfy three properties. The CDF satisfies all of them.

(i) Invertible

- For **continuous densities** $p(x)$, the CDF is **strictly increasing**
- Hence, the inverse F_θ^{-1} exists

(ii) Differentiable

$\frac{d}{dx}F_\theta(x) = p_\theta(x)$

- Since $p_\theta(x)$ is smooth, the CDF is differentiable

(iii) Tractable Jacobian

In 1-D, the Jacobian is just:

$\left|\frac{dz}{dx}\right| = p_\theta(x)$

✅ Cheap and exact to compute

---

4. Sampling via Inverse CDF

Sampling procedure:

1. Sample:

   z∼Uniform[0,1]z \sim \text{Uniform}[0,1]z∼Uniform[0,1]

2. Apply inverse:

   x=Fθ−1(z)x = F_\theta^{-1}(z)x=Fθ−1(z)

This is known as **inverse transform sampling**.

✅ Hence, sampling is exact and efficient.

---

5. Universality of the CDF Flow

Key universality idea

Let:

- FXF_XFX: CDF of data distribution p(x)p(x)p(x)
- FZF_ZFZ: CDF of target distribution pZ(z)p_Z(z)pZ(z)

Then:

x→FXu→FZ−1zx \xrightarrow{F_X} u \xrightarrow{F_Z^{-1}} zxFXuFZ−1z

This composition:

- Maps **any smooth p(x)p(x)p(x)**
- To **any smooth pZ(z)p_Z(z)pZ(z)**

✅ Therefore, **CDFs are universal 1-D normalizing flows**.

---

6. Why This Does NOT Directly Solve Higher-Dimensional Problems

- The CDF is **scalar-valued**
- No unique ordering exists in Rn\mathbb{R}^nRn
- Multivariate CDFs are not invertible in general

➡ This motivates **special flow architectures** for higher dimensions.

---

7. Practical Parameterizations of 1-D CDFs

Since exact CDFs are rarely available, we approximate them using:

(i) Mixture CDFs

- **Gaussian mixture CDF**
- **Mixture of logistics**
- Smooth, flexible, analytically integrable

(ii) Monotone Neural Networks (UMNN)

- Neural networks constrained to be **strictly monotonic**
- Represent valid CDFs
- Used in modern flow models

(iii) Piecewise Spline Flows

- Piecewise linear / quadratic / rational splines
- Monotonic by construction
- Fast and expressive (used in Neural Spline Flows)

---

8. One-Line Exam Summary

In one dimension, the cumulative distribution function (CDF) defines a universal normalizing flow, since it is invertible, differentiable, and maps any continuous distribution to a uniform distribution, allowing exact likelihood computation and sampling via its inverse.

---

9. Intuition (Very Important)

The CDF "flattens" any distribution into a uniform one.
Normalizing flows generalize this flattening idea from 1-D to high dimensions.

CDF explained in very simple language

Think of the **CDF** as a way to answer this question:

"How much of the probability lies to the *left* of a value?"

What is CDF?

CDF stands for **Cumulative Distribution Function**.

For a random variable xxx:

CDF at x=Probability that the value is ≤x\text{CDF at } x = \text{Probability that the value is } \le xCDF at x=Probability that the value is ≤x

---

Simple definition (plain English)

The CDF tells us how likely it is that a value is less than or equal to a given number.

Example 1: Heights

Suppose:

- x=170x = 170x=170 cm
- CDF(170) = 0.65

This means:

65% of people are shorter than or equal to 170 cm

So the CDF:

- Always starts at **0**
- Always ends at **1**
- Keeps increasing

Example 2: Dice (easy intuition)

Imagine a fair die.

- CDF(1) = Probability(die ≤ 1) = 1/6
- CDF(3) = Probability(die ≤ 3) = 3/6 = 0.5
- CDF(6) = Probability(die ≤ 6) = 1

☞ The CDF **adds up probabilities step by step**.

Example 3: Continuous case (important for ML)

For continuous data (like images, audio):

The CDF is:

F(x)=∫−∞xp(t) dtF(x) = \int_{-\infty}^{x} p(t)\,dtF(x)=∫−∞xp(t)dt

In simple words:

Add up all the probability density from −∞-\infty−∞ to xxx

Key properties (easy to remember)

✅ CDF values are **between 0 and 1**
✅ CDF is **always increasing**
✅ CDF at very small value ≈ 0
✅ CDF at very large value ≈ 1

Very important intuition

CDF tells "how far along" the distribution you are.

- Left end → 0
- Middle → around 0.5
- Right end → 1

Why CDF is useful in normalizing flows

The magic fact:

If you apply the CDF to any continuous data, the result becomes Uniform[0,1]

So:

$z = F(x) \sim \text{Uniform}[0,1]$

This is why CDF works as a **perfect 1-D normalizing flow**.

One-line exam answer

The CDF gives the probability that a random variable is less than or equal to a given value, i.e., it accumulates probability from the left up to that point.

One-line intuition (best to remember)

CDF = "How much probability has accumulated so far?"

PDF vs CDF (Simple Explanation)

1. Core Idea (in one sentence each)

- **PDF (Probability Density Function):**
  Tells **how dense or likely** values are *around* a point.
- **CDF (Cumulative Distribution Function):**
  Tells **how much probability has accumulated** up to a point.

---

2. Think of a Road Analogy (Very Intuitive)

Imagine you are walking on a road from left to right.

- **PDF** = how crowded the road is at your exact position
- **CDF** = how many people you have passed **so far**

---

3. PDF (Probability Density Function)

What it means

- Describes **local likelihood**
- Shows **where data is concentrated**

Definition (continuous case)

$p(x) = \text{PDF}$

Important points

- PDF value **is NOT a probability**
- It can be greater than 1
- Probability comes from **area under the curve**

$P(a \le x \le b) = \int_a^b p(x)\,dx$

Simple words

PDF shows the shape of the distribution.

---

4. CDF (Cumulative Distribution Function)

What it means

- Adds up probability **from the left up to x**

Definition

$F(x) = P(X \le x) = \int_{-\infty}^{x} p(t)\,dt$

Important points

- CDF is **always between 0 and 1**
- Always **increasing**
- Final value is always **1**

Simple words

CDF tells "how much probability has already happened."

---

5. Relationship Between PDF and CDF

Very important formulas (exam gold)

- **CDF is the integral of PDF**

$F(x) = \int_{-\infty}^{x} p(t)\,dt$

- **PDF is the derivative of CDF**

$p(x) = \frac{d}{dx} F(x)$

---

6. Side-by-Side Comparison

Aspect	PDF	CDF
Full name	Probability Density Function	Cumulative Distribution Function
Tells	Local density	Total probability till x
Range	$[0,\infty)$	$[0,1]$
Shape	Can go up and down	Always increasing
Probability	Area under curve	Value itself
Math relation	Derivative of CDF	Integral of PDF

---

7. Simple Numerical Example

Suppose:

- $F(3) = 0.7$

This means:

70% of data values are ≤ 3

If the PDF at $x=3$ is high:

Many values occur around 3

---

8. Why This Matters in Normalizing Flows

- **PDF** → used to compute likelihoods
- **CDF** → used to transform data to Uniform[0,1]
- In 1-D:

z=F(x)∼Uniform[0,1]z = F(x) \sim \text{Uniform}[0,1]z=F(x)∼Uniform[0,1]

This is why **CDF is a perfect 1-D flow**.

---

9. Very Short Exam Answer (2–3 lines)

The PDF describes the probability density at a given point, while the CDF gives the cumulative probability that a random variable is less than or equal to that point. The CDF is the integral of the PDF, and the PDF is the derivative of the CDF.

---

10. One-Line Intuition to Remember

PDF = "How dense here?"
CDF = "How far have we come?"

Flow Families in Normalizing Flows

Normalizing flows model a complex distribution by applying a sequence of **invertible transformations** with tractable Jacobians. Different **flow families** differ in how the transformation fθf_\thetafθ is chosen.

---

1. Affine Flow Family

Core idea

Apply a **simple affine (linear + shift)** transformation to part of the input while keeping the rest fixed.

Basic form (1-D intuition)

$z = f(x) = ax + b \quad \text{with } a \neq 0$

- Inverse exists:

$x = \frac{z - b}{a}$

- Jacobian determinant:

$\left|\frac{dz}{dx}\right| = |a|$

Multidimensional version (Coupling layers)

Used in **RealNVP, Glow**.

Split input:

$x = (x_1, x_2)$

Transform:

$$\begin{aligned} z_1 &= x_1 \\ z_2 &= x_2 \odot \exp(s(x_1)) + t(x_1) \end{aligned}$$

- $s(\cdot)$, $t(\cdot)$: neural networks
- Jacobian is **triangular**
- Log-determinant is cheap:

$\log|\det J| = \sum s(x_1)$

Properties

✅ Invertible
✅ Exact likelihood
✅ Fast sampling
❌ Limited expressiveness per layer
→ Needs **many layers** to model complex distributions

Examples

- RealNVP
- Glow
- NICE (additive version)

Exam summary

Affine flows use simple scale-and-shift transformations with triangular Jacobians, enabling efficient likelihood computation but limited expressivity.

---

2. CDF-Based Flow Family

Core idea

Use **Cumulative Distribution Functions (CDFs)**, which are **monotonic and invertible**, as flow transformations.

Key theoretical fact (1-D)

For any continuous random variable xxx:

$z = F(x) = \int_{-\infty}^{x} p(t)\,dt \quad \Rightarrow \quad z \sim \text{Uniform}[0,1]$z=F(x)=∫−∞xp(t) dt⇒z∼Uniform[0,1]

Thus, **CDFs are universal 1-D flows**.

Transformation

$x \xrightarrow{F_X} u \xrightarrow{F_Z^{-1}} z$xFXuFZ−1z

- Maps any smooth p(x)p(x)p(x) to any smooth p(z)p(z)p(z)
- Inverse exists if CDF is strictly monotonic

Practical parameterizations

Since true CDFs are unknown, we approximate them using:

(i) Mixture CDFs

- Gaussian mixture CDF
- Logistic mixture CDF

✅ Smooth and expressive
❌ Evaluation can be costly

(ii) Monotone Neural Networks (UMNN, DSF)

- Neural networks constrained to be monotonic
- Integral of positive functions

✅ Theoretically elegant
❌ Slower than affine flows

Properties

✅ Universal in 1-D
✅ Exact likelihood
❌ Hard to scale directly to high dimensions

Exam summary

CDF-based flows exploit the fact that the CDF of any continuous distribution is invertible and maps data to a uniform distribution, making them universal 1-D normalizing flows.

---

3. Spline Flow Family

Core idea

Approximate an unknown monotonic function using **piecewise polynomial splines** that are invertible and flexible.

Form

Define a monotone function using **bins**:

- Piecewise linear
- Piecewise quadratic
- Piecewise **rational-quadratic** (most popular)

Each bin is parameterized by:

- Widths
- Heights
- Slopes

Rational Quadratic Spline (RQS)

Used in **Neural Spline Flows**.

Properties:

- Smooth
- Strictly monotonic
- Closed-form inverse
- Cheap Jacobian determinant

Transformation (conceptual)

z=f(x)={spline1(x),x∈bin1spline2(x),x∈bin2⋮z = f(x) = \begin{cases} \text{spline}_1(x), & x \in \text{bin}_1 \\ \text{spline}_2(x), & x \in \text{bin}_2 \\ \vdots \end{cases}z=f(x)= ⎩ ⎨ ⎧spline1(x),spline2(x),⋮x∈bin1x∈bin2

Properties

✅ Highly expressive
✅ Fewer layers than affine flows
✅ Fast and stable
❌ Slightly more complex implementation

Examples

- Neural Spline Flow (NSF)
- Rational Quadratic Neural Flow

Exam summary

Spline flows use monotonic piecewise polynomial functions to create flexible, invertible transformations with efficient Jacobian computation.

---

4. Comparison at a Glance (Conceptual)

Flow family	Expressiveness	Speed	Universality	Popularity
Affine	Low per layer	Very fast	No	High
CDF-based	Very high (1-D)	Medium	Yes (1-D)	Medium
Spline	High	Fast	Approx.	Very high

---

5. Big Picture Insight

- **Affine flows** → simple, scalable, need depth
- **CDF-based flows** → theoretically universal in 1-D
- **Spline flows** → best practical trade-off between flexibility and efficiency

---

One-line takeaway (to remember)

Affine flows are simple, CDF flows are universal, and spline flows are the most expressive in practice.

The Coupling Layer Idea

Motivation

In normalizing flows, we need transformations that are:

- ✅ **Invertible**
- ✅ **Differentiable**
- ✅ **Have a tractable Jacobian determinant**

A **coupling layer** is a clever design that satisfies all three while still being expressive.

Core Idea (in simple words)

Split the input into two parts.
Leave one part unchanged, and use it to transform the other part.

Because half the variables remain unchanged, inversion and Jacobian computation become easy.

Mathematical Formulation

Let the input be:

x=(x1,x2)x = (x_1, x_2)x=(x1,x2)

where x1x_1x1 and x2x_2x2 are two disjoint subsets of dimensions.

Forward Transformation (Affine Coupling)

y1=x1y2=x2⊙exp⁡(s(x1))+t(x1)\begin{aligned} y_1 &= x_1 \\ y_2 &= x_2 \odot \exp(s(x_1)) + t(x_1) \end{aligned}y1y2=x1=x2⊙exp(s(x1))+t(x1)

- s(⋅)s(\cdot)s(⋅), t(⋅)t(\cdot)t(⋅): neural networks
- ⊙\odot⊙: element-wise multiplication
- x1x_1x1 is unchanged
- x2x_2x2 is scaled and shifted based on x1x_1x1

Why Is This Invertible?

Because:

x1=y1x2=(y2−t(y1))⊙exp⁡(−s(y1))\begin{aligned} x_1 &= y_1 \\ x_2 &= (y_2 - t(y_1)) \odot \exp(-s(y_1)) \end{aligned}x1x2=y1=(y2−t(y1))⊙exp(−s(y1))

✅ The inverse exists and is computationally cheap.

Jacobian Determinant (Key Advantage)

The Jacobian matrix is **triangular**:

J=[I0∂y2∂x1diag(exp⁡(s(x1)))]J = \begin{bmatrix} I & 0 \\ \frac{\partial y_2}{\partial x_1} & \text{diag}(\exp(s(x_1))) \end{bmatrix}J=[I∂x1∂y20diag(exp(s(x1)))]

So:

log⁡∣det⁡J∣=∑s(x1)\log |\det J| = \sum s(x_1)log∣detJ∣=∑s(x1)

✅ No expensive matrix determinant
✅ Computation is O(n)O(n)O(n)

Why Is This Called a "Coupling" Layer?

Because:

- One part of the input **controls** how the other part is transformed
- The two parts are **coupled** through s(x1)s(x_1)s(x1) and t(x1)t(x_1)t(x1)

Expressiveness Issue (and the Fix)

Problem

- In one coupling layer, **half of the variables don't change**

Solution

- Stack multiple coupling layers
- **Alternate splits or permute dimensions** between layers

Over multiple layers:

- Every dimension gets transformed
- The overall mapping becomes highly expressive

Variants of Coupling Layers

1. Additive Coupling (NICE)

y2=x2+t(x1)y_2 = x_2 + t(x_1)y2=x2+t(x1)

- Simple
- No scaling
- Less expressive

---

2. Affine Coupling (RealNVP, Glow)

$y_2 = x_2 \odot \exp(s(x_1)) + t(x_1)$

- More expressive
- Most commonly used

---

3. Spline Coupling (Neural Spline Flows)

- Replace affine transform with a **monotone spline**
- Much more flexible
- Still tractable

Where Coupling Layers Are Used

- **NICE**
- **RealNVP**
- **Glow**
- **Neural Spline Flows**

They are the **backbone of modern flow architectures**.

One-Line Intuition

Change half the variables while freezing the other half—this makes inversion and likelihood computation easy.

Very Short Exam Answer (2–3 lines)

A coupling layer splits the input into two parts, keeps one part unchanged, and uses it to transform the other part. This results in an invertible transformation with a triangular Jacobian, allowing efficient computation of the log-determinant and inverse.

Key Takeaway (to remember)

Coupling layers trade per-layer simplicity for overall expressiveness by stacking many layers.

Standard Autoencoder (AE)

- The **encoder** converts input xxx into a **fixed point** in latent space:

  z=e(x)z = e(x)z=e(x)

- The **decoder** reconstructs the input from this point:

  x^=d(z)\hat{x} = d(z)x^=d(z)

- The same input always gives the same latent vector zzz.
- **Problem:** Latent space can be irregular and discontinuous, so generating new data is difficult.

Variational Autoencoder (VAE)

- The encoder does **not output a single point**.
  Instead, it outputs a **probability distribution**:

  qϕ(z|x)=N(μx,σx)q_\phi(z|x) = \mathcal{N}(\mu_x, \sigma_x)qϕ(z|x)=N(μx,σx)

- A latent vector zzz is **sampled** from this distribution.
- The decoder reconstructs:

  x^=d(z)\hat{x} = d(z)x^=d(z)

Key Differences (Simplified)

1. Stochastic Latent Space

- **AE:** Latent vector is deterministic.
- **VAE:** Latent vector is **random (stochastic)** due to sampling.
- ✅ This **forces continuity** in the latent space (nearby points produce similar outputs).

2. KL Divergence Penalty

- VAE adds a **KL divergence loss**:

  KL(qϕ(z|x) || p(z))KL(q_\phi(z|x) \,||\, p(z))KL(qϕ(z|x)||p(z))

- This pushes the learned distribution close to:

  p(z)=N(0,I)p(z) = \mathcal{N}(0, I)p(z)=N(0,I)

- ✅ This **regularizes** the latent space and prevents overfitting.

Generation Phase (Most Important Difference)

- **AE:** No clear way to generate new data.
- **VAE:**
  1. Sample z∼N(0,I)z \sim \mathcal{N}(0, I)z∼N(0,I)
  2. Decode zzz to generate a **new data sample**

One-Line Summary (For Exams)

- **AE** learns to compress and reconstruct data.
- **VAE** learns a **smooth and regular latent space**, enabling **data generation**.

Stochasticity in VAE means latent variables are sampled from a distribution rather than fixed, which forces the decoder to learn smooth and continuous mappings, enabling stable interpolation, regularization, and realistic data generation.

One-Sentence Intuitive Summary

Stochastic latent space turns isolated dots into smooth regions, making the model generative and meaningful.

1. Entropy (H(p)) — “How surprising is the data?”

Simple meaning

Entropy measures uncertainty or surprise.

It tells us **how hard it is to guess the outcome**.

Imagine this situation

Case 1: Fair coin

- Heads: 50%
- Tails: 50%

You are **not sure** what will come next → **high uncertainty**

✅ **High entropy**

Case 2: Biased coin

- Heads: 99%
- Tails: 1%

You can almost **predict the result** → **low uncertainty**

✅ **Low entropy**

So, in very simple words

Distribution	Meaning	Entropy
Uniform (all outcomes equally likely)	Very unpredictable	High
Peaked (one outcome dominant)	Very predictable	Low

Formal definition (just for reference)

$H(p) = -\sum_x p(x)\log p(x)$

📌 **Meaning:**
Average number of **surprise bits** needed to describe events from distribution **p**.

---

2. Cross-Entropy (H(p, q)) — “Using the wrong model”

Simple meaning

Cross-entropy measures how inefficient it is when you use the wrong probability model.

Data comes from **true distribution p**,
but you *assume* it follows **distribution q**.

Intuitive example

- True data (**p**):
  - Cats: 90%
  - Dogs: 10%
- Your assumed model (**q**):
  - Cats: 50%
  - Dogs: 50%

☞ You are **bad at predicting** the data
☞ You need **more bits to encode** it

✅ This extra cost is measured by **cross-entropy**

Definition (reference only)

$H(p,q) = -\sum_x p(x)\log q(x)$

📌 **Meaning:**
Average surprise when:

- data is from **p**
- but you encode it using **q**

---

3. Key Properties (Very Important)

✅ Cross-entropy is always ≥ entropy

$H(p,q) \ge H(p) H(p, q) \ge H(p) H(p,q) \ge H(p)$

Why?

- Using the **right model** is always best
- Using a **wrong model wastes extra bits**

✅ When are they equal?

$H(p,q) = H(p) \quad \text{if and only if} \quad q = p H(p,q) = H(p) \quad \text{if and only if} \quad q = p$

📌 Meaning:

- If your model **perfectly matches reality**
- There is **no extra cost**

---

4. One-Line Intuition (Best for exams)

- **Entropy:** how unpredictable the data really is
- **Cross-entropy:** how bad your predictions are when your model is wrong

---

5. Ultra-Short Exam Answers

Entropy

Entropy measures the average uncertainty or surprise in a probability distribution.

Cross-Entropy

Cross-entropy measures the extra cost incurred when data from distribution p is encoded using an incorrect distribution q.

---

6. Simple Analogy (Easy to remember)

- **Entropy:**
  "How difficult is the exam?"
- **Cross-entropy:**
  "How difficult is the exam **plus** how badly you prepared?"